

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Mohan Kumar J	Reg. No.:	192424060
Section / Batch:	2024 [AI&DS]	Date:	20-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I Mohan Kumar J (192424060) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Mohan Kumar

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Healthcare Data Sorting System Using Merge Sort

1. Introduction

Healthcare systems generate a large amount of patient data every day. Patient records may contain information such as patient ID, priority level, admission time, department, and treatment details.

When many patients are admitted at the same time, the hospital needs to arrange records efficiently. Emergency patients may need to be considered before routine patients. If two patients have the same priority, their admission time can be used to determine their order.

The proposed system uses Merge Sort to sort patient records based on priority and admission time.

The main objectives are:

- Reduce patient waiting time.
- Give higher priority to urgent patients.
- Maintain accurate and consistent ordering.
- Process large volumes of healthcare data efficiently.
- Maintain stable ordering when records have equal values.

2. Healthcare Case and Requirements

The healthcare data sorting system receives patient records and arranges them according to priority and admission time.

Patient	Priority	Admission Time
P001	3	09:30
P002	1	08:45
P003	2	09:10
P004	1	08:30
P005	3	08:50

A smaller priority number represents a higher priority. The expected sorting order is: higher-priority patients first; if priority is equal, earlier admission time first; if both values are equal, the original order is maintained.

The system must provide accurate sorting, stable ordering, consistent results, efficient processing of large datasets, and proper handling of duplicate priority values.

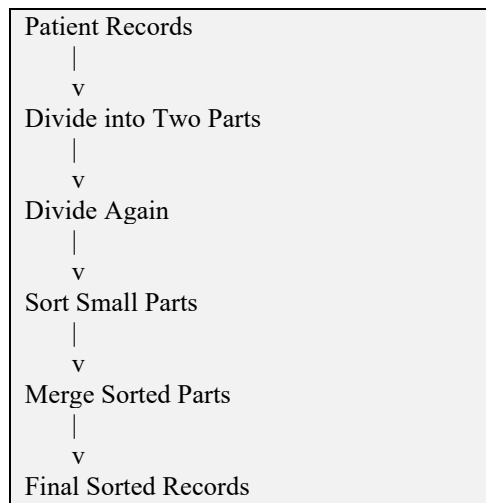
3. Merge Sort and Divide-and-Conquer Approach

Merge Sort is a sorting algorithm based on the Divide and Conquer technique.

Divide: The complete list is divided into two smaller parts.

Conquer: Each smaller part is recursively sorted using Merge Sort.

Combine: The sorted parts are merged together to produce one sorted list.



For n patient records, the list is repeatedly divided until each part contains only one record. The individual parts are then merged in sorted order.

4. Sorting Criteria and Stability

The healthcare system uses a multi-level comparison rule.

Primary Criterion: Priority

Patients with higher priority are placed first. For example, Priority 1 is processed before Priority 2.

Secondary Criterion: Admission Time

If two patients have the same priority, the patient who was admitted earlier is placed first.

Patient	Priority	Admission
P001	2	09:30
P002	2	08:45

The correct order is P002 followed by P001.

Stability

Merge Sort is a stable sorting algorithm when implemented correctly. If two patient records have the same priority and admission time, their original order is preserved. This is useful because records with equal sorting values should not be unnecessarily rearranged.

5. Step-by-Step Example

Patient	Priority	Admission Time
P001	3	09:30
P002	1	08:45
P003	2	09:10
P004	1	08:30
P005	3	08:50
P006	2	08:20

Step 1: Divide the records into smaller groups until individual records are obtained.

P001 P002 P003 P004 P005 P006

Step 2: Sort and merge the records using priority first and admission time second.

Priority 1: P004 → P002, because 08:30 is earlier than 08:45.

Priority 2: P006 → P003, because 08:20 is earlier than 09:10.

Priority 3: P005 → P001, because 08:50 is earlier than 09:30.

Final sorted records:

Position	Patient	Priority	Admission Time
1	P004	1	08:30
2	P002	1	08:45
3	P006	2	08:20
4	P003	2	09:10
5	P005	3	08:50
6	P001	3	09:30

6. Algorithm and Pseudocode

```
MERGE_SORT(records)

    If the number of records is less than or equal to 1:
        Return records

    Find the middle position

    Divide records into:
        Left half
        Right half

    Sort the left half using MERGE_SORT

    Sort the right half using MERGE_SORT

    Merge the two sorted halves

    Return the merged list
```

During merging, compare two records. Select the record with the smaller priority. If priorities are equal, select the record with the earlier admission time. If both values are equal, select the left-side record first to maintain stability.

7. Python Implementation

```
from dataclasses import dataclass

@dataclass
class Patient:
    patient_id: str
    priority: int
    admission_time: str

def merge(left, right):

    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):

        if (left[i].priority < right[j].priority or
            (left[i].priority == right[j].priority and
             left[i].admission_time <= right[j].admission_time)):

            result.append(left[i])
            i += 1

        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

def merge_sort(records):

    if len(records) <= 1:
        return records

    middle = len(records) // 2

    left = merge_sort(records[:middle])
    right = merge_sort(records[middle:])

    return merge(left, right)

patients = [
    Patient("P001", 3, "09:30"),
    Patient("P002", 1, "08:45"),
    Patient("P003", 2, "09:10"),
    Patient("P004", 1, "08:30"),
```

```

    Patient("P005", 3, "08:50"),
    Patient("P006", 2, "08:20")
]

sorted_patients = merge_sort(patients)

print("Sorted Patient Records:")

for patient in sorted_patients:
    print(
        patient.patient_id,
        patient.priority,
        patient.admission_time
    )

```

Expected Output:

```

Sorted Patient Records:

P004 1 08:30
P002 1 08:45
P006 2 08:20
P003 2 09:10
P005 3 08:50
P001 3 09:30

```

8. Time Complexity and Large Data Analysis

Merge Sort is suitable for healthcare systems because hospitals may need to process thousands or millions of patient records.

At every level, the records are divided into smaller parts and then merged. The number of levels is approximately $\log_2 n$. At each level, all n records are processed during merging.

Therefore, the overall time complexity is:

$O(n \log n)$

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

This predictable performance makes Merge Sort effective for large datasets.

Standard Merge Sort requires additional memory for merging:

Space Complexity = $O(n)$

Although Merge Sort uses additional memory, its predictable $O(n \log n)$ performance and stable behavior make it suitable for large healthcare datasets.

9. Accuracy, Validation, and Test Cases

The system should be tested carefully because incorrect patient ordering could affect healthcare operations.

Test Case	Input Situation	Expected Result
TC01	Different priorities	Higher-priority patients appear first
TC02	Same priority	Earlier admission time appears first
TC03	Same priority and admission time	Original order is preserved
TC04	Empty dataset	Empty result without error
TC05	One patient	Record returned without changes
TC06	Large dataset	Efficient $O(n \log n)$ processing

Validation conditions:

- Priority is correctly ordered.
- Admission time is correctly ordered for equal priorities.
- No patient records are lost.
- No patient records are duplicated.
- Equal records maintain their original order.
- The algorithm produces consistent results.

10. Solution Evaluation and Conclusion

Merge Sort provides an effective solution for the healthcare data sorting problem.

The main advantages are:

- Efficient: $O(n \log n)$ time complexity.
- Stable: Maintains the order of equal records.
- Accurate: Correctly handles multiple sorting criteria.
- Scalable: Suitable for large healthcare datasets.
- Predictable: Best, average, and worst-case time complexity are $O(n \log n)$.
- Suitable for Divide and Conquer: Large datasets are divided into smaller manageable parts.

The use of priority and admission time provides a practical sorting strategy. Higher-priority patients can be placed first, while admission time maintains a fair order among patients with the same priority.

The main limitation of Merge Sort is its additional memory requirement of $O(n)$. However, for large healthcare datasets, the predictable $O(n \log n)$ running time and stable sorting behavior make it a strong choice.

Overall, Merge Sort is an efficient, stable, and reliable algorithm for organizing large healthcare patient records based on priority and admission time. It provides accurate results while maintaining good performance as the amount of patient data increases.

ASSESSMENT-CO3/AT2-CASE STUDY[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)

QUESTIONS

Q1Healthcare Data Sorting System
(Merge Sort)
100 marks

1/1 answered

Q1 Healthcare Data Sorting System (Merge Sort)

100 marks

A healthcare system sorts patient records based on priority and admission time. Analyze how Merge Sort ensures stable and accurate sorting. Identify issues such as large data volume and consistency. Apply divide and conquer techniques. Analyze time complexity and justify its effectiveness.

Your Code

```
1 patients=[
2     ["John",2,"10:30"],
3     ["Alice",1,"09:45"],
4     ["David",3,"11:45"],
5     ["Emma",1,"08:30"],
6     ["Micheal",2,"09:20"],
7     ["Sophia",1,"10:00"]
8 ]
9 def compare(p1,p2):
10     if p1[1]<p2[1]:
11         return True
12     elif p1[1]>p2[1]:
13         return False
14     return p1[2]<p2[2]
15 def merge(left,right):
16     result=[]
17     i=0
18     j=0
19     while i<len(left) and j<len(right):
20         if compare(left[i],right[j]):
21             result.append(left[i])
22             i+=1
23         else:
24             result.append(right[j])
25             j+=1
26     while i<len(left):
27         result.append(left[i])
28         i+=1
29     while j<len(right):
30         result.append(right[j])
31         j+=1
32     return result
33 def merge_sort(arr):
34     if len(arr)<=1:
35         return arr
36     mid=len(arr)//2
37     left=merge_sort(arr[:mid])
38     right=merge_sort(arr[mid:])
39     return merge(left,right)
40 print("HEALTHCARE DATA SORTING SYSTEM")
41 print("-"*25)
42 print("\n Patient Records Before Sorting \n")
43 print("Name\t Priority \t Admission time")
44 for patient in patients:
45     print(patient[0], "\t\t", patient[1],patient[2])
```

 Submitted

Input

stdin input

Output

Name	Priority
Admission time	
John	2 10:30
Alice	1 09:45
David	3 11:45
Emma	1 08:30